EECS 498 · AASE · FALL 2026

Where we are, and starting on Aider

LAB03 · OCT 5–6, 2026

Today’s schedule

1. What we are changing
2. Setup: Codex and Aider
3. Learning Aider with Codex
4. Designing skills for Aider

What we are changing

YOU WERE INTERESTED. WE WANT TO SEE WHAT WE CAN DO.

Frontier tools, used well

The most common thing you want to learn: using the tools *well*.

About half

of your answers on what to learn

> *"how to push the limits of frontier models and harnesses"*

> *"more realistic (modern) coding agent workflows"*

> *"the workflow of how actual engineers use it"*

WHAT WE ARE CHANGING

More focus on the tools

From today you work with *Codex*, and labs teach you to use it well.



Everyone uses Codex

On Luna, with your course key



Labs are hands-on

Your laptop, real code



We teach using it well

Learn the code, then design

WHAT WE ARE CHANGING

Big codebases, starting with Aider

Small models meant small repos. Better tools mean the *real* Aider.

Until now

Aider on a small model

Repos you built from scratch

Sized to what the model could handle

From today

Codex on Luna

The real Aider: 20,000 lines of Python

Plus 12,000 lines of tests

Today you start adding Claude Code features to it. Two of you proposed exactly this.

WHAT WE ARE CHANGING

Learn, then design, then build



1. Learn

Codex teaches you the code



2. Design

You make the decisions



3. Build

Small steps, good prompts

Skip step 1 and the model makes every decision in step 2 for you.

What we are still figuring out

Most of getting good is *use*, and use is hard to teach.

How to teach the tools

Beyond "use them a lot"

What learning looks like

When the model writes the code

What the projects require

Details later this term

We will work these out with you. The form at the end is the first step.

Setup: Codex and Aider

Codex on the Toolkit

its.umich.edu/computing/ai/codex-classroom

Follow the ITS page from the top. Your key already exists.

Model: `gpt-6-luna`.

Already using Codex

Keep your own setup. Add the course key as a *profile*.

```
# add to ~/.codex/config.toml
[model_providers.toolkit]
name = "UM GPT Toolkit"
base_url = "https://api.toolkit.umgpt.umich.edu/v1"
env_key = "UMGPT_TOOLKIT_API_KEY"
```

```
# new file ~/.codex/toolkit.config.toml
model = "gpt-6-luna"
model_provider = "toolkit"
model_reasoning_effort = "medium"
```

`codex --profile toolkit` uses the course key. Plain `codex` stays on your own account.

Put your course key in UMGPT_TOOLKIT_API_KEY. Tested on Codex 0.160.

Your copy of Aider

1. Sent out like your other course repos
2. The EGM skills are already inside
3. Open Codex at the repository root

Elephant, Goldfish, peanuts and the rest, in `.agents/skills/`, where Codex looks.

Learning Aider

SKILLS FIRST, THEN PEANUTS, THEN YOUR OWN QUESTIONS

The five skills in your repo

Skill	What it does	What it writes
peanuts	Maps a codebase, bottom up	A README per folder, `PEANUTS.md`
elephant	A design conversation, no code	A design doc, `ELEPHANT.md`
goldfish	Fresh reviewers test the doc	`GOLDFISH.md`
egm-implement	Builds from an approved doc	Code, `IMPLEMENT.md`
mean-review	A hostile code review	`MEAN-REVIEW.md`

Read each `SKILL.md` in `.agents/skills/` before you run it, with its walkthrough in materials/egm/walkthroughs/.

A map with peanuts

```
Run the peanuts skill on aider/, excluding aider/website. Write one README at a time and stop after each.
```

1. Read the README peanuts wrote
2. Ask Codex to teach you that part
3. Iterate with Codex until the README is clear and right
4. Approve it, then the next one

FOLLOW-UP

"What does base_coder.py do that the other coders inherit? Show me where."

ITERATE

"Check aider/coders/README.md against the code. Rewrite anything unclear or wrong."

Codex as a teacher

Ask about *anything*. Codex reads the code to answer.

THE CODE
"Where does Aider build the prompt it sends?"

THE DESIGN
"Why edit blocks instead of tool calls?"

THE TESTS
"How do the tests run without a real model?"

WHAT IF
"What breaks if I delete repomap.py?"

Keep asking until you could explain it without Codex.

Designing skills for Aider

Skills in Aider

Aider has no skill loader. Today you *design* one.

Aider today

You add SKILL.md by hand

The whole body, every request

Until you drop it

Claude Code and Codex

Only the description, every request

The model asks for the body

Loaded when it matches

The rest of today

1. Learn where prompts get built
2. Elephant: decide how it should work
3. Build one small step, with your design

No one-shot prompts. You will not finish today, and that is fine.

Questions?